DEPARTMENT OF
NETWORKED SYSTEMS
AND SERVICES

Webbiztonság

VIHIAC07 – IT biztonság, 2026

Dr. Gazdag András

CrySyS Lab, BME

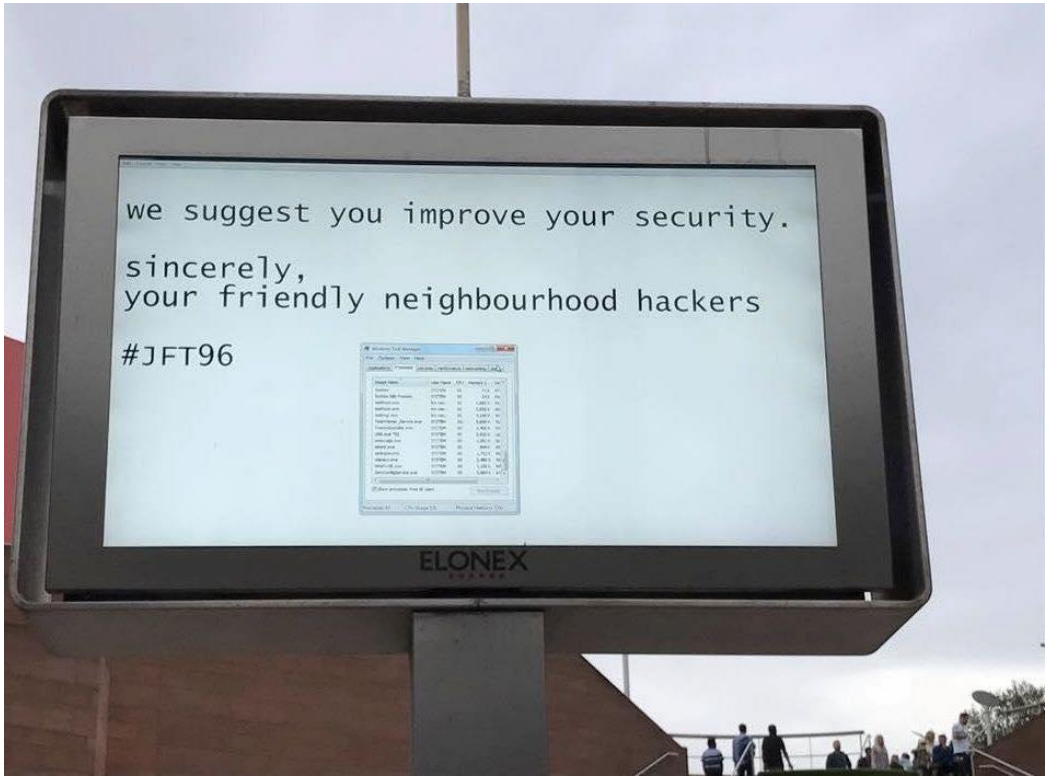
agazdag@crysys.hu

www.crysys.hu

MŰEGYETEM 1782

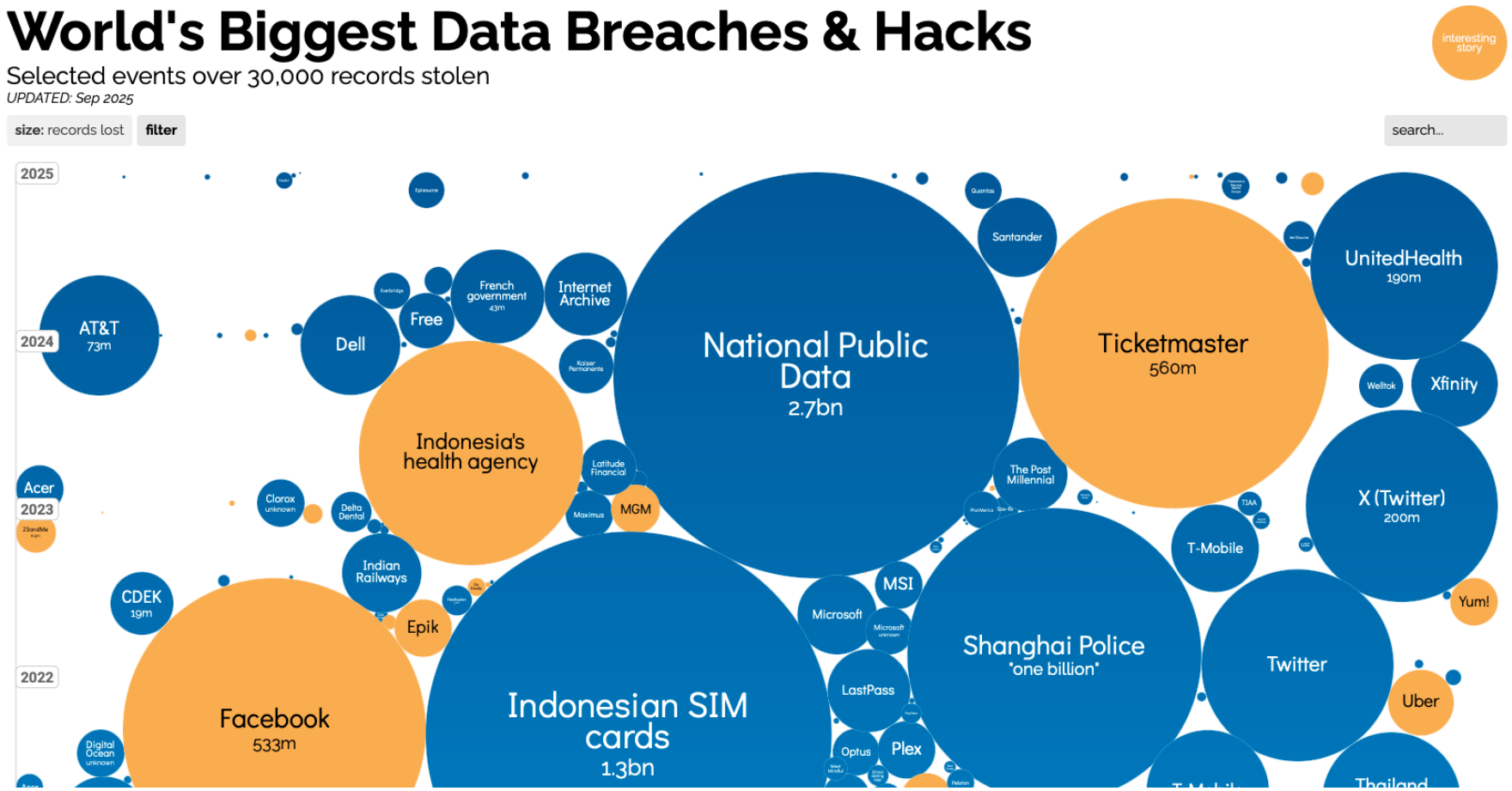
Tartalom

- **Webbiztonság**
  - Incidensek
  - OWASP Top 10
  - Érdekes támadások



Webbiztonsági incidensek

Information is Beautiful – 2025



Adatszivárgások költsége iparáganként

Here is a breakdown of some of the costliest industries for data breaches in 2024:

Industry	Average 2023 Breach Cost	Average 2024 Breach Cost	% Increase
Healthcare	$10.93 million	$9.77 million	-10.6%
Finance	$5.9 million	$6.08 million	+3%
Industrial	$4.73 million	$5.56 million	+17.5%
Technology	$4.66 million	$5.45 million	+17%
Energy	$4.78 million	$5.29 million	+10.7%
Pharmaceuticals	$4.82 million	$5.1 million	+5.1%
Professional services	$4.47 million	$5.08 million	+13.6%

Egy adatszivárgás lehetséges hatása

OWASP – TOP 10

OWASP

- Open Worldwide Application Security Project (OWASP)
  - Világszerte működő nonprofit szervezet, amely a szoftverbiztonság javítására összpontosít
  - Küldetése a szoftverbiztonság láthatóvá tétele, hogy az egyének és szervezetek megalapozott döntéseket hozhassanak
- Mindenki szabadon részt vehet az OWASP-ban
  - minden anyaguk ingyenes és nyílt licenc alatt érhető el
- “Does not endorse or recommend commercial products or services, allowing our community to remain vendor neutral with the collective wisdom of the best minds in software security worldwide”

OWASP Top 10

- A 10 legkritikusabb webalkalmazás-biztonsági kockázat listája
- A jelenlegi verzió 2025-ben jelent meg

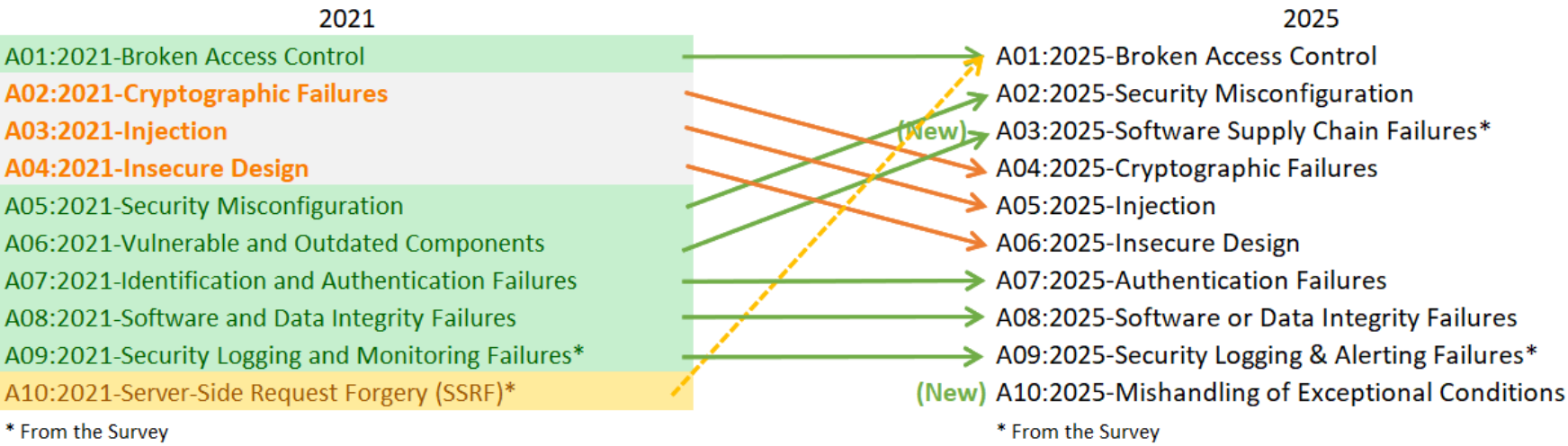
- Nem egy ellenőrzőlista!
  - Csak prioritási sorrendet mutat

- https://owasp.org/Top10/2025

OWASP Top 10 – 2025

- A01:2025 - Broken Access Control
- A02:2025 - Security Misconfiguration
- A03:2025 - Software Supply Chain Failures
- A04:2025 - Cryptographic Failures
- A05:2025 - Injection
- A06:2025 - Insecure Design
- A07:2025 - Authentication Failures
- A08:2025 - Software or Data Integrity Failures
- A09:2025 - Security Logging and Alerting Failures
- A10:2025 - Mishandling of Exceptional Conditions

OWASP Top 10 – 2025-ös trendek



Ne állj meg 10-nél!

- Több száz probléma befolyásolhatja egy webalkalmazás *általános biztonságát*, ahogy az az „OWASP Developer's Guide” és az „OWASP Cheat Sheet” sorozatban bemutatásra kerül
  - https://github.com/OWASP/DevGuide
  - https://owasp.org/www-project-cheat-sheets

- Az „OWASP Testing Guide” iránymutatást nyújt a webalkalmazásokban és API-kban található sérülékenységek hatékony felderítéséhez.
  - https://owasp.org/www-project-web-security-testing-guide
  - Ezek nagyon ajánlott olvasmányok fejlesztők számára!

- Az „OWASP DevSecOps Guideline” arra összpontosít, hogyan valósíthatunk meg biztonságos pipeline-t és ismerteti ehhez a legjobb gyakorlatokat és eszközöket
  - https://owasp.org/www-project-devsecops-guideline

A01:2025 Broken Access Control

A01:2025 Broken Access Control

- A hozzáférés-védelem biztosítja, hogy a felhasználók csak a számukra engedélyezett műveleteket végezhessék el
  - A hibák jellemzően a következőkhöz vezethetnek
    - » jogosulatlan információszerzés
    - » az összes adat módosítása vagy megsemmisítése
    - » a felhasználó hatáskörén kívüli művelet végrehajtása

- Kapcsolódó fontosabb CWE-k:
  - CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
  - CWE-201: Exposure of Sensitive Information Through Sent Data
  - CWE-918: Server-Side Request Forgery (SSRF)
  - CWE-352: Cross-Site Request Forgery (CSRF)
  - CWE-22 Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')

A01:2025 Broken Access Control

- Példa: A támadó ”kézzel” megnyit a böngészőben URL-eket: az admin oldal csak adminisztrátorok számára kellene, hogy elérhető legyen, viszont ez csak az URL elrejtésével van megvalósítva:
  - `https://example.com/app/`*`getappInfo`*
  - `https://example.com/app/`*`admin_getappInfo`*

- A hozzáférés-védelem csak akkor hatékony, ha az megbízható szerver oldali kódban vagy serverless API-kban van megvalósítva, ahol a támadó nem tudja módosítani azokat.
  - A nyilvánosan elérhető erőforrásokon kívül alapértelmezés szerint tilts minden hozzáférést.
  - Tiltsd le a webszerver könyvtárlistázását, és gondoskodj arról, hogy a rejtett fájlok (pl. .git) és a biztonsági mentések ne legyenek elérhetők a webes gyökérkönyvtárban.
  - Naplózd a hozzáférés-védelmi szabálysértéseket, és értesítsd az adminisztrátorokat szükség esetén (pl. ismétlődő hibák esetén).

A02:2025 Security Misconfiguration

A02:2025 Security Misconfiguration

- Konfigurációs hiba akkor áll fenn, amikor egy rendszert, alkalmazást vagy felhőszolgáltatást biztonsági szempontból helytelenül konfigurálnak, ezzel sérülékenységeket teremtve.

- Sérülékeny lehet, ha:
  - Felesleges funkciók vannak engedélyezve vagy telepítve (pl. szükségtelen portok, szolgáltatások, oldalak, fiókok, vagy jogosultságok).
  - Az alapértelmezett fiókok és jelszavak engedélyezve vannak.
  - Az alkalmazás egyes részein hiányos a biztonsági megerősítés (hardening), vagy a felhőszolgáltatások jogosultságai nincsenek megfelelően beállítva.
  - Nincs központi konfiguráció a túlzottan részletes hibaüzenetek kezelésére.
    - » A hibakezelés stack trace-eket vagy más, feleslegesen részletes információt tartalmazó hibaüzeneteket jelenít meg a felhasználóknak.

A02:2025 Security Misconfiguration

- Példa: Az alkalmazásszerver konfigurációja lehetővé teszi, hogy részletes hibaüzenetek – például stack trace-ek – jussanak el a felhasználókhoz.
  - Ez érzékeny információkat vagy rejtett hibákat fedhet fel, például olyan komponensverziókra utalva, amelyekről ismert, hogy sérülékenyek.

- Biztonságos deploy folyamatokat kell megvalósítani, beleértve:
  - Egy megismételhető megerősítési (hardening) folyamat, amely lehetővé teszi egy megfelelően lezárt környezet gyors és egyszerű létrehozását.
    - » A fejlesztési, tesztelési (staging) és éles környezeteket azonos módon kell konfigurálni, minden környezetben eltérő hitelesítési adatokkal.
    - » Ezt a folyamatot automatizálni kell, hogy minimalizálják az új környezet beállításához szükséges munkát.
  - Törekedjünk a minimális platformra, felesleges funkciók, komponensek, dokumentáció vagy teszt adatok nélkül.
    - » A nem használt funkciókat és keretrendszereket el kell távolítani, vagy eleve ne legyenek telepítve.

A02:2025 Security Misconfiguration – Content Discovery

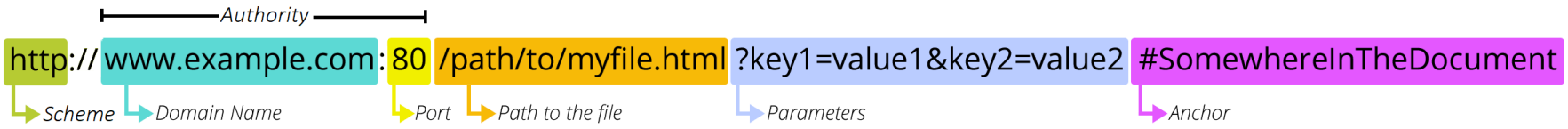
- A támadó első lépése: a *támadási felület maximalizálása*

- Manuális tesztelés HTTP proxyval
  - Fizetős: Burp Suite, open source: Zed Attack Proxy
- Web spider használata
  - Minden erőforrás bejárása hivatkozások alapján
  - Általában korlátolt: nem hajt végre JS-t
- Könyvtárlistázás kihasználása
  - Ha webszerver listázza egy könyvtár összes fájlját
    - » Érdemes kikapcsolni!
- Véletlenszerű találgatás
  - Tipikusan használt fájlnevek alapján

Same origin policy
és
Cross Origin Resource Sharing

A02:2025 Security Misconfiguration

URL-ek felépítése



- *Scheme*: a protokollt jelöli
- *Authority*: a domain és a port, kettősponttal elválasztva
- *Path*: az erőforráshoz vezető út a webszerveren
- *Parameters*: kulcs/érték párok listája, & jellel elválasztva
- *Anchor*: egy "könyvjelzőt" jelöl az erőforráson belül (más néven fragment identifier)

+1 opcionális: felhasználónév az authority rész előtt

https://developer.mozilla.org/en-US/docs/Learn/Common_questions/Web_mechanics/What_is_a_URL

Kliens oldali biztonsági modell

Origin = scheme + domain + port kombinációja

Alapszabály: **origin = biztonság határa**

Same Origin Policy

Egy weboldal csak a saját origin-jéből (séma, domain, port) érhet el erőforrásokat korlátozás nélkül. A más origin-ből származó erőforrásokra hozzáférés-védelmi szabályok vonatkoznak.

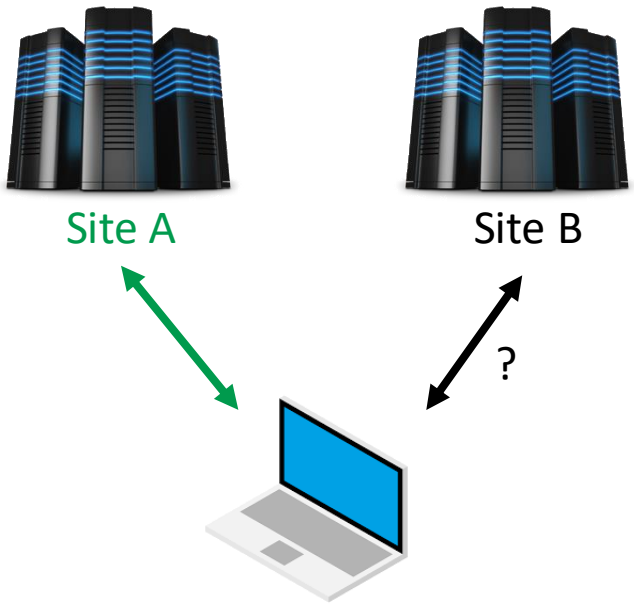
https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy

Same Origin Policy – hozzáférés típusok

- Cross-origin embedding: minden origin-ről megengedett
  - Példák: JavaScript (the `<script>` tag), CSS (`<link…>`), képek (`<img>`), `<audio>`, `<video>`, `<iframe>`, …
- Cross-origin write (sending): minden origin-ről megengedett
  - Példák: linkek, átirányítások, form beküldések
  - E nélkül az oldalak csak saját magukra hivatkozhatnának
  - A CSRF és a clickjacking ettől még lehetséges (azok a SOP hatáskörén kívülre esnek)
- *Cross-origin read (receiving): csak azonos origin-ről megengedett!*
  - De általában beágyazással megkerülhető

Cross-origin Resource Sharing (CORS)

- HTTP alapú hozzáférés-védelem az adatküldéshez és fogadáshoz



- Az egyszerű kérés mindig közvetlenül elküldve:
  - `GET`
  - `HEAD`
  - `POST` – (bizonyos `Content-Types`)

- A bonyolultabb kérések előtt először egy *preflight* kérés kerül kiküldésre:
  - A kérés nem kerül közvetlenül elküldésre (`POST`), `PUT`, `DELETE`, és egyéb HTTP metódusok esetén
  - Először egy *előzetes (preflight)* kérés kerül elküldésre az `OPTIONS` metódussal, a következő fejlécet várva a válaszban:

```
Access-Control-Allow-Origin: http://www.one.site.com
```

A03:2025 Software Supply Chain Failures

A03:2025 Software Supply Chain Failures

- A szoftver ellátási lánc hibái olyan zavarok vagy kompromittálódások, amelyek a szoftver fejlesztési, terjesztési vagy frissítési folyamatában keletkeznek.
  - Ezeket gyakran harmadik fél kódjában, eszközeiben vagy más függőségekben lévő sérülékenységek vagy rosszindulatú módosítások okozzák.

- Valószínűleg sérülékeny vagy, ha:
  - Nem követed nyomon az összes felhasznált komponens verzióját (mind kliens-, mind szerver oldalon), beleértve a közvetlen és közvetett(!) függőségeket is.
  - Egy szoftver sérülékeny, már nem támogatott vagy elavult verzióját használod.
    - » Ez magában foglalja az operációs rendszert, a web-/alkalmazásszervert, az adatbázis-kezelő rendszert (DBMS), az alkalmazásokat, az API-kat és az összes komponenst, futtatókörnyezetet és könyvtárat.
  - Nem végzel rendszeres sérülékenységvizsgálatot, és nem iratkozol fel a felhasznált komponensekkel kapcsolatos biztonsági értesítésekre.

A03:2025 Software Supply Chain Failures

- Példa: Egy megbízható beszállító termékét malware-rel kompromittálnak, ami így a frissítések után a mi rendszereinkbe is eljut.
  - Ennek talán leghíresebb példája a 2019-es SolarWinds elleni támadás ami ~18,000 további szervezet feltöréséhez vezetett

- Egy patch menedzsment folyamatnak kell kidolgozva lennie:
  - Központilag kezeli és generálja a Software Bill of Materials-t (SBOM)
  - Nemcsak a közvetlen függőségeket, hanem azok közvetett (tranzitív) függőségeit is nyomon követi
  - A támadási felületet csökkenti a nem használt függőségek, felesleges funkciók, komponensek, fájlok és dokumentáció eltávolításáva

A04:2025 Cryptographic Failures

A04:2025 Cryptographic Failures

- A transport rétegen minden átvitt adatot titkosítani kell.
- Az érzékeny adatokat csak titkosítva szabad tárolni.

- Tipikus hibák:
  - Régebbi kódban elavult vagy gyenge kriptográfiai algoritmusokat vagy protokollok használata
  - Gyenge kulcsokat generálása, vagy kulcsok újra felhasználása, vagy az ajánlott kulcs-frissítések elmulasztása
  - Verziókezelő rendszerbe kulcs bekerülése
  - Titkosítás hiánya, javasolt HTTP header-ök nem megfelelő beállítása

A04:2025 Cryptographic Failures

- Példa: jelszavak sózás nélküli, vagy gyenge hash függvénnyel tárolása
  - Ha a támadó valahogy meg tudja szerezni a jelszó adatbázist, akkor a gyenge módszerrel tárolt jelszavak brute-force támadás célpontjává válhatnak.

- Ajánlás:
  - Az alkalmazás által feldolgozott, tárolt vagy továbbított adatok osztályozása.
    - » Annak meghatározása, hogy mely adatok érzékenyek az adatvédelmi törvények, szabályozási követelmények vagy üzleti igények alapján.
  - A legérzékenyebb kulcsokat hardveres vagy felhő alapú HSM-ben kell tárolni.
  - Mindenhol, ahol csak lehetséges, jól bevált kriptográfiai algoritmus-implementációkat kell használni.

A05:2025 Injection

A05:2025 Injection

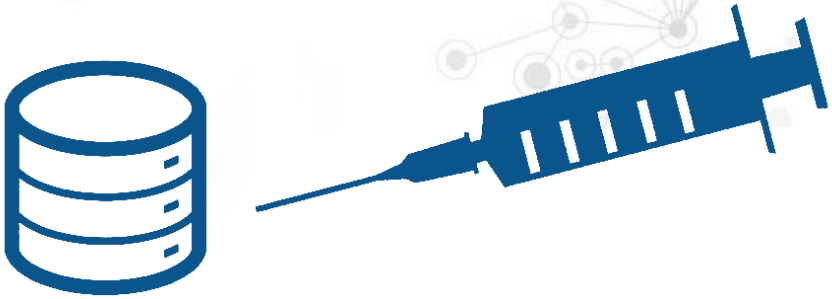
- Ha egy alkalmazás sérülékeny beszúrásos támadásra, az lehetővé teszi, hogy egy nem megbízható felhasználói bemenet kerüljön egy értelmezőhöz (pl. böngésző, adatbázis, parancssor), és az értelmező a bemenet egyes részeit parancsként hajtsa végre.

- Egy alkalmazás sérülékeny lehet, ha:
  - A felhasználó által megadott adatokat az alkalmazás nem validálja, szűri vagy tisztítja
  - Dinamikus lekérdezéseket vagy nem paraméteres hívásokat kontextusérzékeny ellenőrzés (escape) nélkül közvetlenül használ az értelmezőben

A05:2025 Injection

- Példa: egy alkalmazás közvetlenül operációs rendszer parancsnak adja át a felhasználói bemenetet:
  - `String cmd = "nslookup " + request.getParameter("domain"); Runtime.getRuntime().exec(cmd);`
  - Ilyen esetben a támadó a következő bemenetet adhatja meg: *`example.com; cat /etc/passwd`*
    - » Ez a sérülékenység tetszőleges parancs futtatására ad lehetőséget a támadónak

- A beszúrásos támadás megelőzésének legjobb módja az adatok elkülönítése a parancsoktól:
  - Előnyös ilyenkor egy (biztonságos) paraméteres API használata, amely teljes mértékben elkerüli az értelmező használatát



SQL injekció

A05:2025 Injection

SQL

- SQL = Structured Query Language
- Adatbázis lekérdező nyelv relációs adatbázisok számára

ID	NAME	PASSWORD	EMAIL
1	joe	SG2MB45BK	joe@example.com
2	jane	SZ634BT723	jane@example.com
3	john	KJZ245JZ78J	john@example.com
4	jill	FU3489VI76	jill@example.com

```
SELECT * FROM users WHERE name=jill AND password=FU3489VI76
```

- Több dialektusa létezik

SQL injection

- Nagyon gyakori probléma: egy SQL lekérdezése előállítása string műveletekkel **külső** adatok alapján:

```python
def login(db, username, password):
  query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}';"
  result = db.execute(query)
  return result
```

- Művelet elvárt eredménye:

```
SELECT * FROM users WHERE username='john.doe' AND
password='janedoe';
```

- Azonban, ha a támadó a bemenet részeként SQL utasítást is megad, az alapjaiban módosíthatja az eredeti lekérdezést. Például: **`asd' OR 'a'='a`**,

```
SELECT * FROM users WHERE username='asd' OR 'a'='a' AND
password='whatever';
```

Tipikus SQL injection támadási módszerek

```
SELECT * FROM users WHERE username='???' AND password='???';
```

- SQL komment string: #, --, /*
  - DBMS függő a támadás

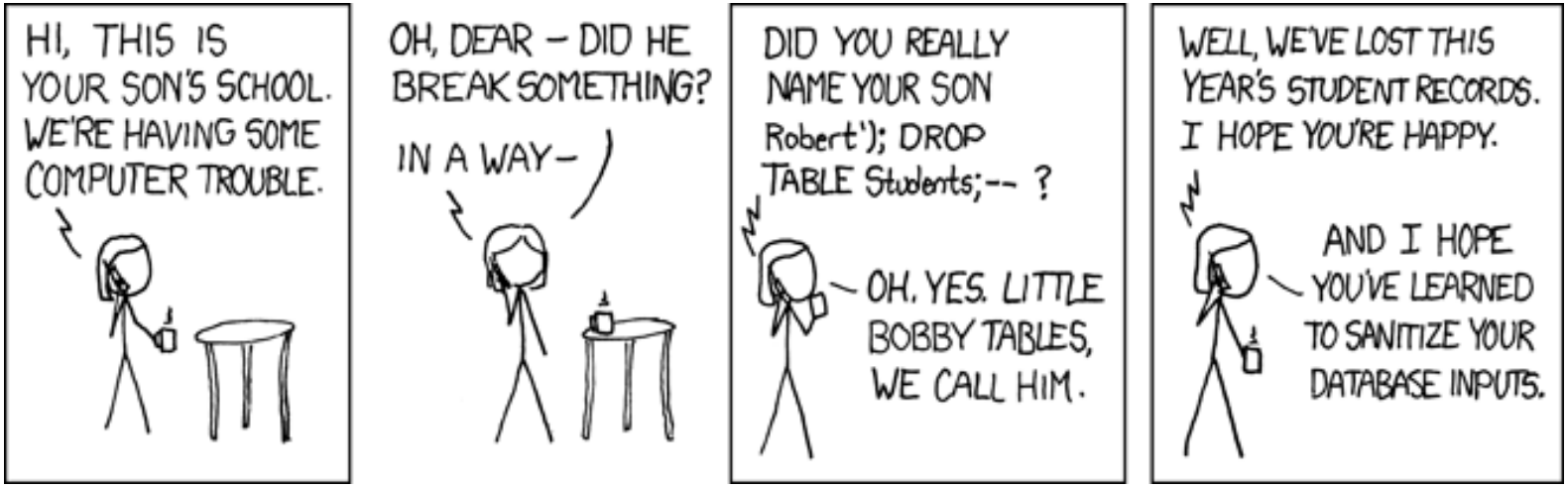
```
username=a' # rest of the query
```

Tipikus SQL injection támadási módszerek

```
SELECT * FROM users WHERE username='???' AND password='???';
```

- Query stacking segítségével több utasítás összefűzhető
  - ; segítségével el lehet választani több utasítást egymástól
  - DBMS és platform függő ez a támadás

```
username=a'; DELETE * from users #
```



SQL injection programok

- De facto sztenderd megoldás SQLMap
- Képes automatikusan kihasználni az egyszerűbb hibákat
- Könnyen bővíthető a komplex esetek kezelésére:
  - Python ”tamper script”-ek segítségével

```
[11:07:01] [INFO] target URL appears to have 3 columns in query
[11:07:01] [INFO] GET parameter 'id' is 'MySQL UNION query (NULL) - 1 to 20 columns'
 injectable
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [
y/N] N
sqlmap identified the following injection points with a total of 25 HTTP(s) requests
:
---
Place: GET
Parameter: id
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Payload: id=1 AND 3362=3362

    Type: error-based
    Title: MySQL >= 5.0 AND error-based - WHERE or HAVING clause
    Payload: id=1 AND (SELECT 9338 FROM(SELECT COUNT(*),CONCAT(0x3a6976743a,(SELECT
(CASE WHEN (9338=9338) THEN 1 ELSE 0 END)),0x3a766b663a,FLOOR(RAND(0)*2))x FROM INFO
RMATION_SCHEMA.CHARACTER_SETS GROUP BY x)a)
```

Védekezés

- Kerülni kell a dinamikus SQL lekérdezéseket
  - Amikor csak lehet statikusan megírt lekérdezéseket kell futtatni
    - » Ilyen lekérdezések nem változnak futásidőben, így nem is sérülékenyek
- A bemeneteket szűrni és ellenőrizni kell
  - A speciális karaktereket escape-elni kell, kikényszeríteni a sztenderd név konvenciók használatát, stb.
- Parameterized statement-ek használata
  - Speciális karakterek jelzik a bemenetek helyét
  - A bemenetek speciális függvények segítségével kerülnek behelyettesítésre
- Állítsunk be megfelelő jogosultságokat az adatbázison
  - Pl: a webszerver felhasználónak csak olvasási joga legyen alapból, ha lehet

SQL injection – Input validáció

- Engedélyezési listák: hatékony megoldás, de nem mindig alkalmazható.

- Tiltólisták: mindig hiányoznak róla elemek! Nem ajánlott!
  - Probléma 1: `DROP -> DRO/**/P`
  - Probléma 2: karakterkódolás (pl. Unicode)
  - Megengedett bemenetek kiszűrése is előfordulhat

- Escaping: érdemes mindig beépített függvényt használni, és nem sajátot implementálni! (Python: `mysql_escape_string()`)

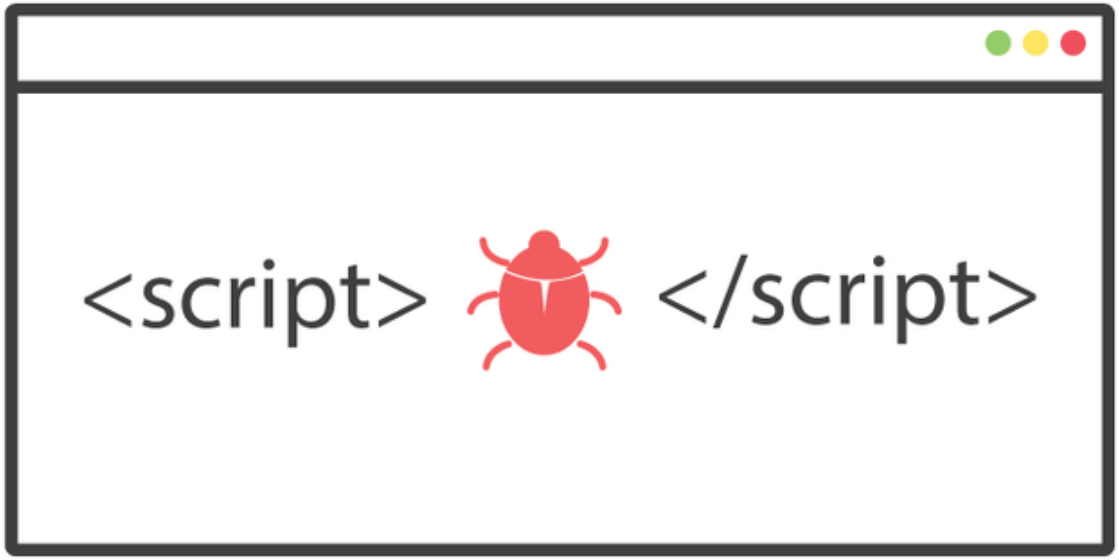
Parameterized statements

- A legtöbb környezetben létezik implementáció hozzá
- A felhasználói inputok úgy kerülnek behelyettesítésre, hogy a DBMS garantáltan nem fogja azokat a kód részeként értelmezni

- Python példa:

```
connection = mysql.connector.connect(...)
cursor = connection.cursor(prepared=True)

sql_query = """SELECT * FROM Users WHERE
               EmailAddr=%s AND Password=%s"""
cursor.execute(sql_query, (username, password))
```



Cross-Site Scripting (XSS)

A05:2025 Injection

Cross Site Scripting

Cross Site Scripting (XSS): a támadó képes JavaScript kódot futtatni egy másik originen belül

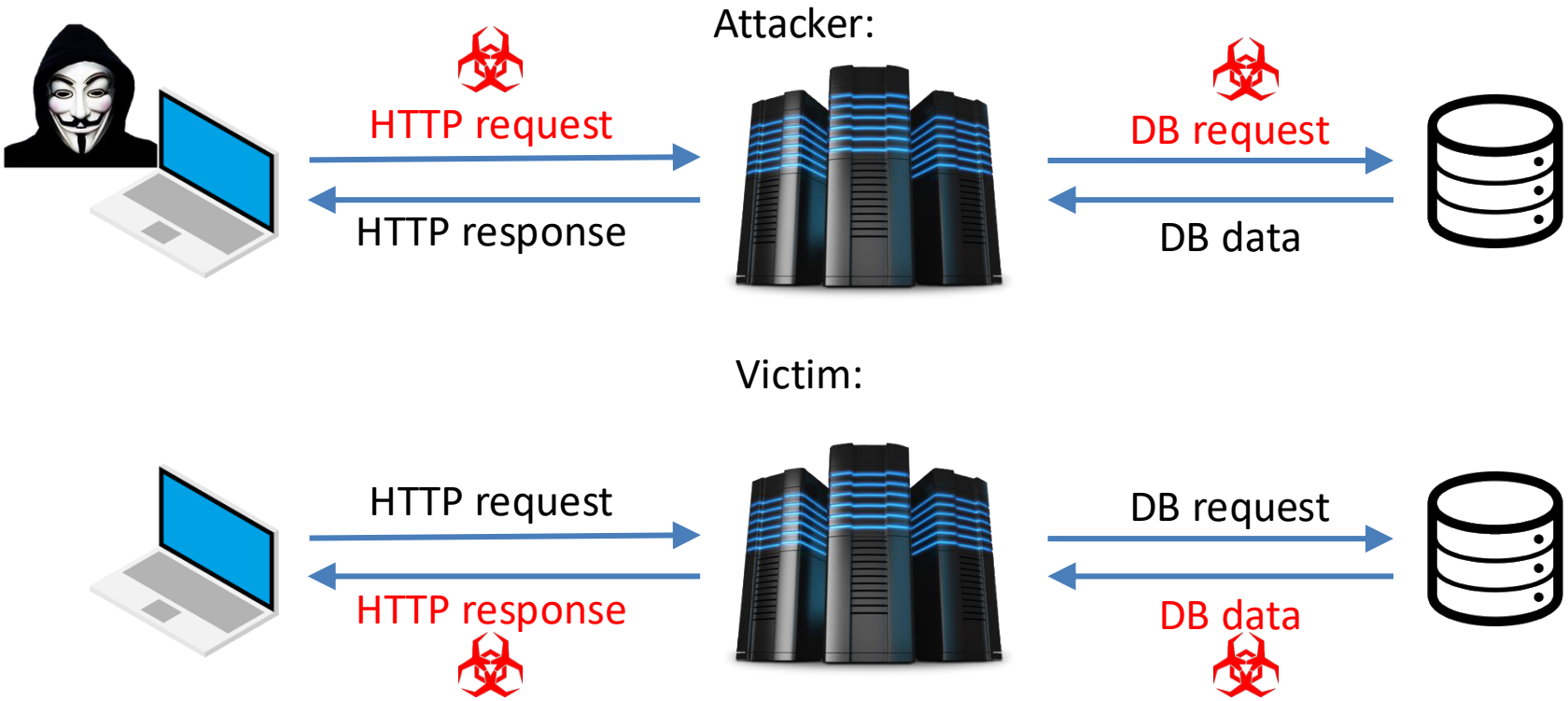
- A legveszélyesebb kliens oldali támadás
- A kártékony kód lényegében bármit megtehet a támadás során

- Több különböző altípusát is megkülönböztetjük a támadásnak, az alapján, hogy:
  - A kártékony kód tárolsára kerül-e a szerver oldalon
  - A HTML tartalom összeállítása hol történik meg

Cross Site Scripting típusai

1. Persistent/Stored XSS

- A kártékony kód a szerver oldalon eltárolásra kerül
- Például: kommentben, üzenetben, felhasználói adatban, stb.
- Trigger: az áldozat meglátogatja a fertőzött oldalt



Cross Site Scripting típusai

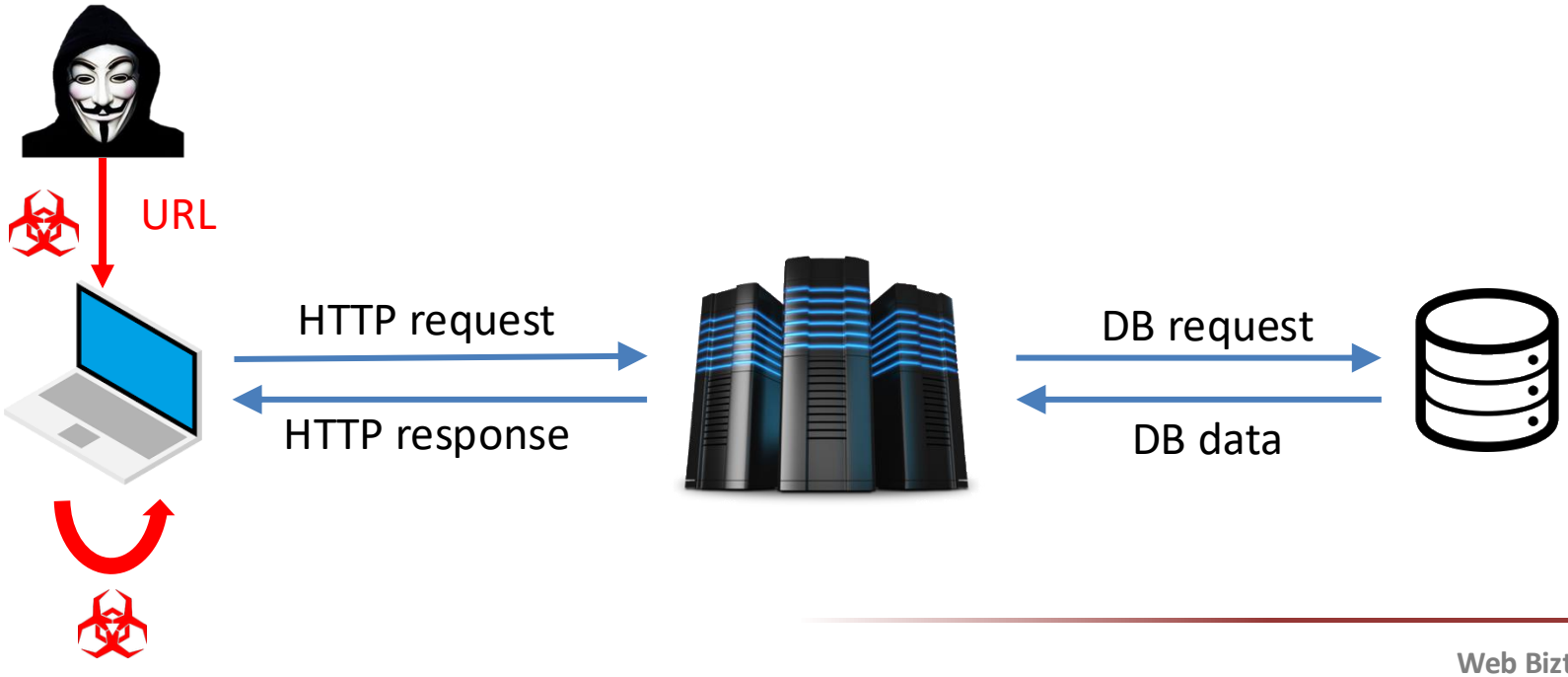
2. Reflected XSS

- A kártékony JS egy GET/POST paraméteren át fertőz
- A szerver oldal felhasználja a paraméter értékét a HTML előállítása során
- Trigger: a felhasználó meglátogatja a sérülékeny oldalt a kártékony URL-en keresztül

Cross Site Scripting típusai

3. DOM alapú XSS

- A támadás tisztán a kliens oldalon történik
- HTML összeállítása a kliens oldalon van megvalósítva (pl: SPA)
  - » `x.innerHTML = attacker_controlled_variable;`
- Trigger: a felhasználó meglátogatja a sérülékeny oldalt a kártékony URL-en keresztül



Cross Site Scripting - megelőzés

- Kívülről származó adatot ellenőrizni kell mielőtt az a HTML-be kerül!
- A *kontextus fontos*! Különböző ellenőrzéseket kell végeni!
  - `<p><?php echo $user_comment; ?></p>`
  - `<img src="…" title="<?php echo $user_title; ?>" >`
  - `<script>n = "<?php echo $user_name; ?>";</script>`
- Tiltó lista alapú megközelítés vagy egyszerű törlés nem elég!
  - Törlés esetén: `<scr<script>ipt>` → `<script>`
  - Egy tiltó lista **sosem** teljes!
- Megoldás HTML elemek és tulajdonságok esetén:
  - HTML entity encoding
  - PHP: `htmlspecialchars()`
- [OWASP XSS Prevention Cheat Sheet]

Cross Site Scripting - védekezés

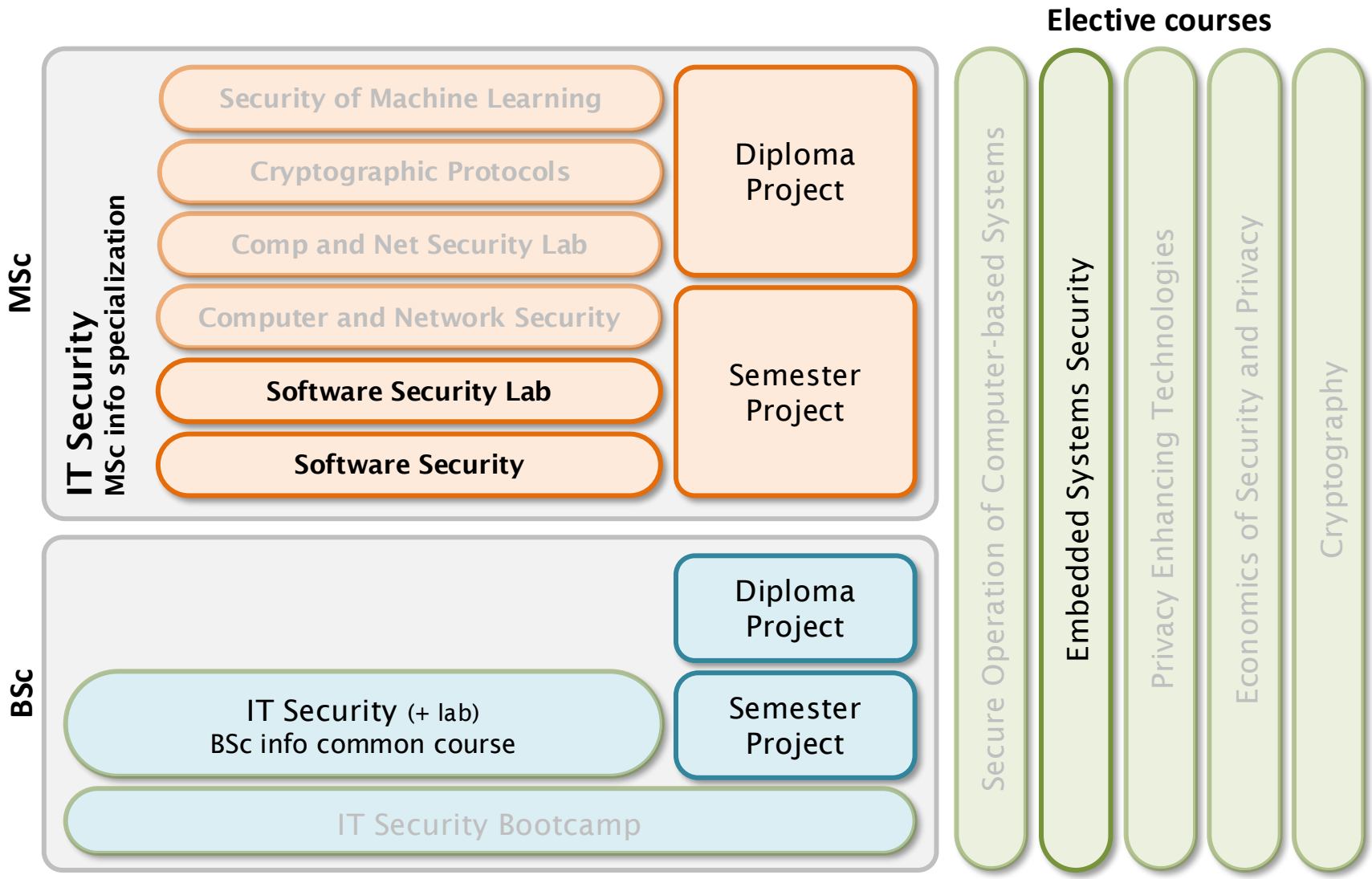
- **HTTP-only sütik**
  - Egy tulajdonsága a sütiknek
  - Nem lehet JS kódból hozzáférni → session lopás lehetetlen XSS-en át

- **Content Security Policy (CSP)**
  - HTTP fejléc
  - Megadja, hogy honnan tölthet be tartalmat a böngésző
  - Egy megadott URL-re jelzi, ha szabálysértés történik
  - Kiindulási elv:
    - » Ne engedjük az inline script lehetőséget (sokszor nehéz megvalósítani)
    - » Ne engedjük az `eval()` függvény használatát
  - További ötletek:
    - » Csak megadott domainről engedjük a scriptek, képek, objektumok betöltését
    - » Adjuk meg, hogy mely oldal ágyazhatja be frame-be a mi oldalunk

Bug Bounty programok

- A böngésző fejlesztők biztonsági stratégiájának a része
- **Cél: a sérülékenységeket a gyártónak adják el és ne a feketepiacon**
- Komolyabb összegeket is lehet keresni
  - Mozilla: ~15000 USD
  - Chrome: <= ~30000 USD
- Pwn2own
  - Évente megrendezett esemény a CanSecWest konferencián
  - Teljes támadást kell bemutatni
  - Nagyon magas díjjak
- Internet Bug Bounty
  - Általános webhez tartozó sérülékenységek, sandbox escape, stb.

IT security education program (BSc, MSc)



more info: http://www.crysys.hu/education/

Hivatkozások

- SQLMap
  - https://github.com/sqlmapproject/sqlmap
- OWASP XSS Prevention Cheat Sheet
  - https://www.owasp.org/index.php/XSS_(Cross_Site_Scripting)_Prevention_Cheat_Sheet
- Content Security Policy:
  - http://www.w3.org/TR/CSP/
- Burp Suite
  - http://portswigger.net/burp/
- Zed Attack Proxy:
  - http://owasp.org/index.php/OWASP_Zed_Attack_Proxy_Project

Hivatkozások

- Shreeraj Shah, Founder & Director, Blueinfy Solutions
  - HTML5 Top 10 Threats Stealth Attacks and Silent Exploits; Blackhat EU 2012
- https://venturebeat.com/2018/03/05/wordpress-now-powers-30-of-websites
- https://developers.slashdot.org/story/17/10/29/0441205/why-do-web-developers-keep-making-the-same-mistakes
- https://www.hpe.com/us/en/insights/articles/the-owasp-top-10-is-killing-me-and-killing-you-1710.html
- https://codecurmudgeon.com/wp/sql-injection-hall-of-shame
- https://www.csoonline.com/article/2130877/data-breach/the-biggest-data-breaches-of-the-21st-century.html
- https://www.openhub.net/p/firefox/analyses/latest/languages_summary
- https://blog.stoneriverelearning.com/top-5-programming-languages-used-in-web-development

Hivatkozások

- Mozilla bug bounty:
  - https://www.mozilla.org/en-US/security/bug-bounty
- Apple Security bounty
  - https://developer.apple.com/security-bounty
- Chrome bug bounty:
  - https://www.google.com/about/appsecurity
- Internet Bug Bounty:
  - https://hackerone.com/internet-bug-bounty

Ellenőrző kérdések

- Hogyan épül fel egy URL?
- Mik a leggyakoribb web biztonsági problémák?
- Mi a célja az OWASP projektnek?
- Általában mi az első lépése egy támadónak egy szerver ellen?
- Mi az általános probléma injection típusú támadásoknál?
- Mutasson egy példát egy SQL injection támadásra!
- Hogyan kell védekezni egy SQL injection ellen?

Ellenőrző kérdések

- A kliens oldalon hogy definiáljuk a biztonsági határt?
- Mi az a Same Origin Policy?
- Mit limitál a Same Origin Policy egy XMLHttpRequest elküldésekor?
- Mi az a XSS támadás?
- Milyen típusai vannak az XSS támadásnak?
- Hogyan lehet védekezni XSS támadással szemben?
- Mi az a Content Security Policy?